

ISL-ViT-Tiny — A Compact Vision Transformer for Indian Sign Language Recognition

Preliminary Model: Architecture, Implementation, Training and Evaluation

Table of Contents

- 1. Executive Summary
- 2. Problem Statement and Design Goals
- 3. Proposed System Architecture
- 4. Dataset
- 5. A Measurement Problem in the Benchmark
- 6. Evaluation Protocols
- 7. Data Pipeline
- 8. Deep Learning Architecture
- 9. Algorithm and Implementation
- 10. Experimental Setup
- 11. Results
- 12. Convergence Analysis
- 13. Ablation Studies
- 14. Error Analysis
- 15. Observations
- 16. Discussion
- 17. Limitations and Threats to Validity
- 18. Conclusions and Future Work
- 19. Reproducibility Appendix

1. Executive Summary

This report documents a preliminary Vision Transformer for isolated word-level Indian Sign Language (ISL) recognition, designed to be small enough for smart-glasses deployment. The model — **ISL-ViT-Tiny** — is a factorised spatio-temporal ViT operating on hand and face crops rather than full frames.

Headline figures.

Property	Value
Parameters	3.80 M
Compute	0.824 GMACs / clip
Model size (INT8)	≈ 3.8 MB
Latency (desktop CPU, single clip)	12 ms
Input	8 frames × 3 streams × 64×64 RGB

Property	Value
Vocabulary	50 (development) / 262 (full) signs

Accuracy depends enormously on the evaluation protocol, and quantifying that dependence turned out to be the most substantive result of this work:

Protocol	What it removes	INCLUDE-50	INCLUDE-263
random-video	nothing	95.7 %	94.5 %
official	nothing	95.3 %	—
take-group	near-duplicate takes	46.5 %	29.3 %
session-disjoint	signer / room / lighting	35.8 %	22.1 %

The conventional random split reproduces published INCLUDE numbers (~94–95 %). The same model, trained identically, scores **35.8 %** once near-duplicate takes and shared recording sessions are excluded from the test set. Roughly **49 percentage points** of the standard benchmark is attributable to near-duplicate takes and a further **11 points** to shared session conditions.

Principal conclusion for the modelling effort. Every capacity-increasing lever tested was flat or negative — tripling the token count changed nothing, and ImageNet initialisation is worth only +6.3 points. The model is **data-limited, not capacity-limited**, which sets the direction for subsequent work.

2. Problem Statement and Design Goals

2.1 Task

Isolated sign language recognition (ISLR): given a short video clip containing a single ISL sign, predict the corresponding word from a fixed vocabulary. This is the classification building block for a continuous recognition system.

2.2 Design constraints

#	Constraint	Rationale
C1	Must be a genuine Vision Transformer	Specified requirement; patch-embedding + self-attention throughout
C2	Small enough for smart glasses	Target ≤ 10 M parameters, ≤ 1 GMAC, INT8-quantisable
C3	Clip-in / word-out, extendable to streaming	Sliding-window inference with a <no-sign> class
C4	Honest evaluation	Reported numbers must reflect generalisation, not memorisation

Constraint **C4** was elevated to a first-class design goal after the analysis in §5.

2.3 Why a factorised architecture

A naive video ViT attends jointly over all patches of all frames. For T frames, S streams and P patches per crop, joint attention costs $O((T \cdot S \cdot P)^2)$. Factorising into a spatial encoder (within a crop) followed by a temporal encoder (across crops) reduces this to $O(P^2) + O((T \cdot S)^2)$.

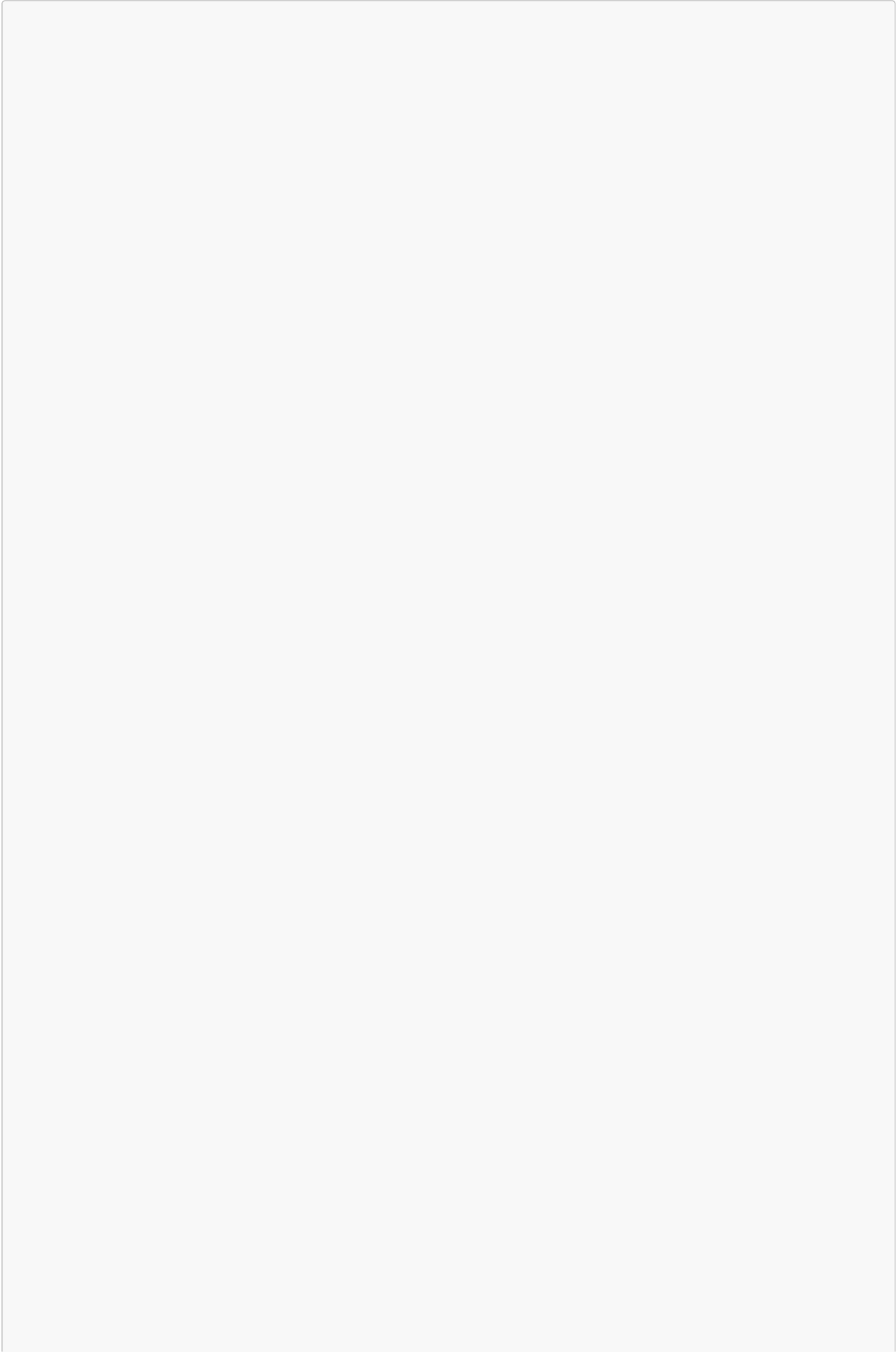
With $T=8$, $S=3$, $P=17$:

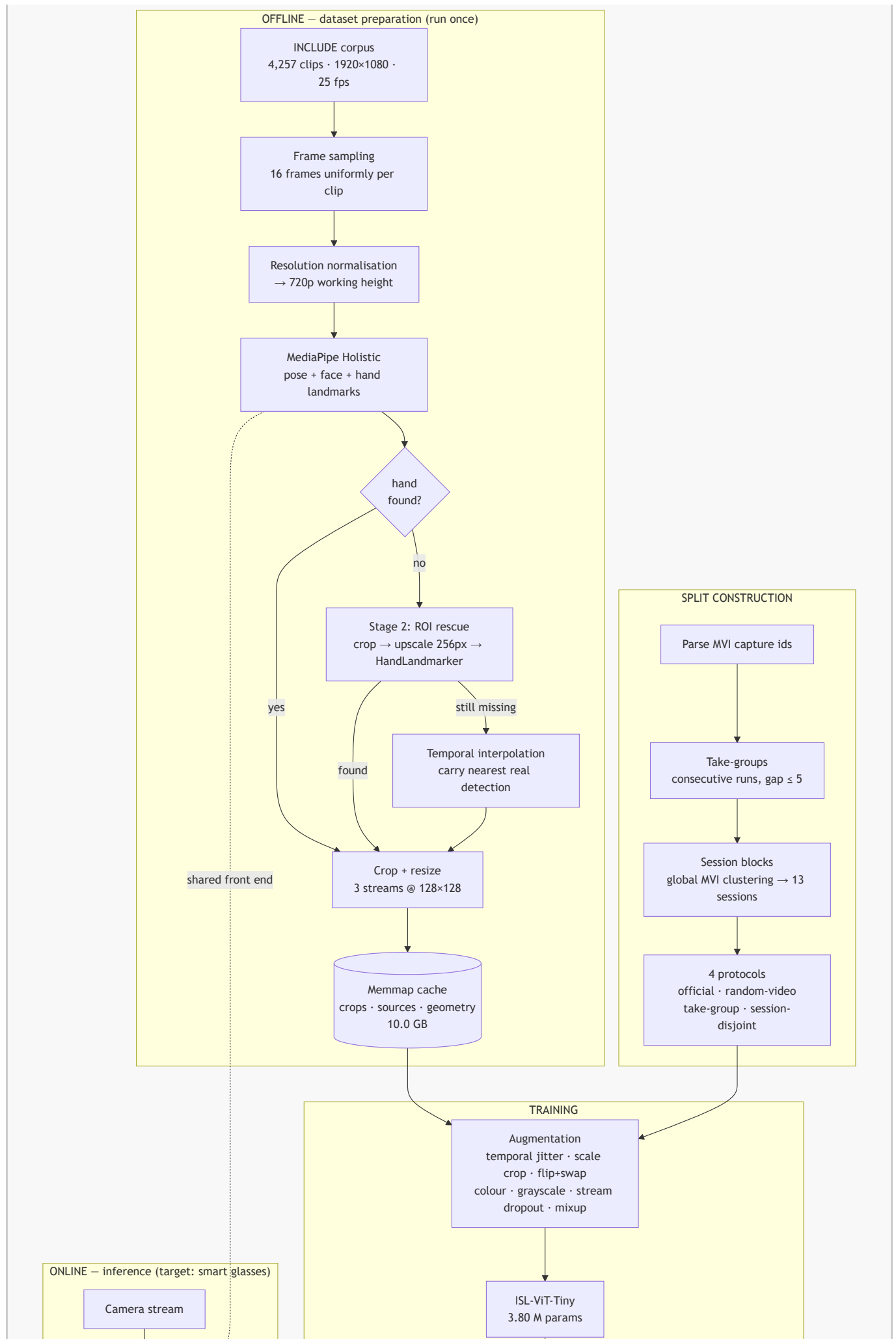
Scheme	Attention token count	Relative cost
Joint spatio-temporal	408	1.00×
Factorised (adopted)	17 then 25	≈ 0.02×

This is what makes the model fit an edge compute budget at all.

3. Proposed System Architecture

The end-to-end system separates an offline data-preparation path from an online inference path. Both share the same crop-extraction front end, which is what allows the cached training representation to match what the deployed model sees.





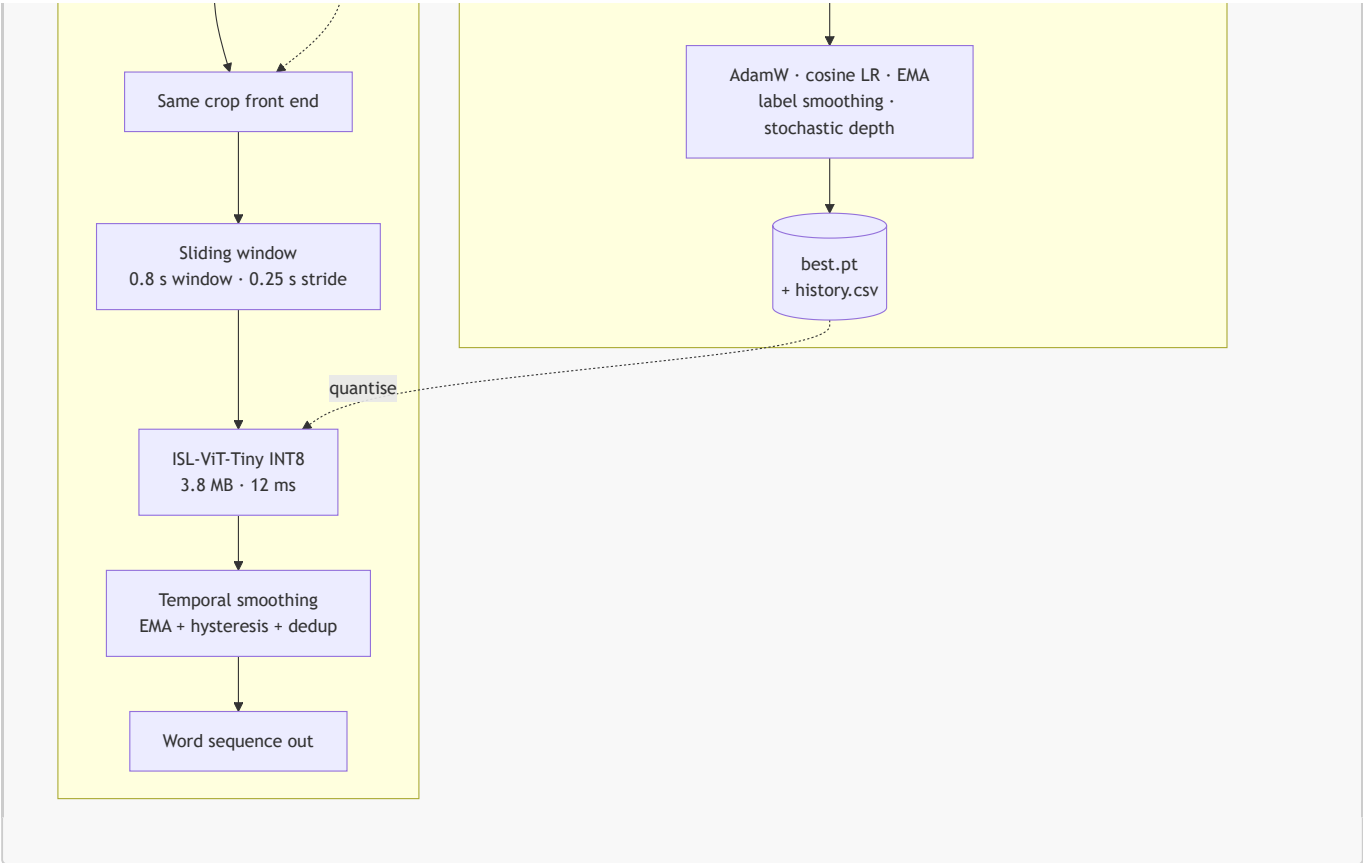


Figure 3.1 — Overall system architecture. Solid arrows are data flow; dashed arrows indicate component reuse between the offline and online paths. Components in the **ONLINE** subgraph marked as target are specified but not yet implemented (see §18).

3.1 Design rationale for the crop front end

The single most consequential architectural decision is that **the model never sees a full frame**. Three streams are extracted per timestep — left hand, right hand, face — and the transformer sees only those.

Motivation	Effect
Background removal	The studio wall, furniture and framing cannot act as class cues
Signer-body removal	Clothing and body shape are largely excluded
Token economy	≈ 10× fewer tokens than full-frame at equal spatial detail
Resolution efficiency	Hand occupies ~10 % of frame width; cropping recovers that detail

Because cropping discards *where* the hands are — and sign location is a defining linguistic parameter — the box geometry is fed back in as an explicit feature (see §8.4).

4. Dataset

4.1 Source

INCLUDE (Sridhar et al., ACM MM 2020), obtained from Zenodo record 4010759, with metadata from the HuggingFace mirror.

Property	Value
Clips	4,257 (unique, after de-duplication)
Classes	262 word-level signs
Semantic categories	15
Native resolution	1920 × 1080
Frame rate	25 fps
Clip duration	≈ 2–5 s (50–120 frames)
Recording location	Chennai, Tamil Nadu (single studio)
Licence	CC-BY-4.0

4.2 Class statistics

Statistic	Clips per class
Minimum	4
Median	15
Mean	16.2
Maximum	22

This is the defining property of the problem. A median of 15 clips per class across 262 classes is an extreme low-data regime for a Vision Transformer, which lacks the convolutional inductive biases that make CNNs sample-efficient. Every choice in §9 follows from this.

4.3 Category distribution

Category	Clips	Category	Clips
Adjectives	788	Jobs	225
People	513	Clothes	198
Places	380	Greetings	190
Home	379	Means of Transportation	186
Society	324	Pronouns	168
Days and Time	298	Animals	161
Colours	222	Electronics	140
—	—	Seasons	85

4.4 A data-integrity issue found and corrected

An earlier processing run had transcoded part of the corpus to 480p and deleted the 1080p originals. The affected clips were **not** randomly distributed:

Category	1080p available	480p only
Adjectives	207	581 (74 %)
Home	309	70 (18 %)
Pronouns	0	168 (100 %)
All others	2,922	0

50 classes were entirely 480p and 212 entirely 1080p, with no class mixed. Encoding quality would therefore have been a *perfect* predictor for those 50 labels, and the model could have learned compression artefacts instead of signs. Nine archives (10.5 GB) were re-downloaded to restore uniform 1080p sources across all 4,257 clips before any reported experiment was run.

5. A Measurement Problem in the Benchmark

5.1 Two benchmarks in one metadata file

The shipped HuggingFace parquets contain 5,250 rows over 4,257 unique clips. Naively loading all three splits produces 277 clips appearing in both "train" and "test". The cause is not corruption: the file stacks the **full-263 split** (`include_50 == False`) on top of the **INCLUDE-50 split** (`include_50 == True`). Each benchmark is internally clean; filtering on the flag is mandatory.

View	Unique clips	train / val / test	Internal overlap
<code>include_50 == False</code>	4,257	3101 / 340 / 816	0
<code>include_50 == True</code>	943	675 / 77 / 191	0

5.2 The corpus is organised as back-to-back takes

Each clip filename carries a camera capture id (`MVI_xxxx`). Sorting these within a class reveals a strongly clustered structure:

```
Class "1. Dog": 2978, 2979, 2980 | 3002, 3003, 3004 | 3028, 3029, 3030 |
                3059, 3060, 3061 | 3085, 3086, 3087 | 4146, 4147, 4148 | 8560,
                8561
```

Quantitatively, over all within-class consecutive id gaps:

Gap	Count	Interpretation
1–2	2,907	Consecutive takes, same sitting
3–19	19	(essentially empty)

Gap	Count	Interpretation
≥ 20	1,063	A different recording session

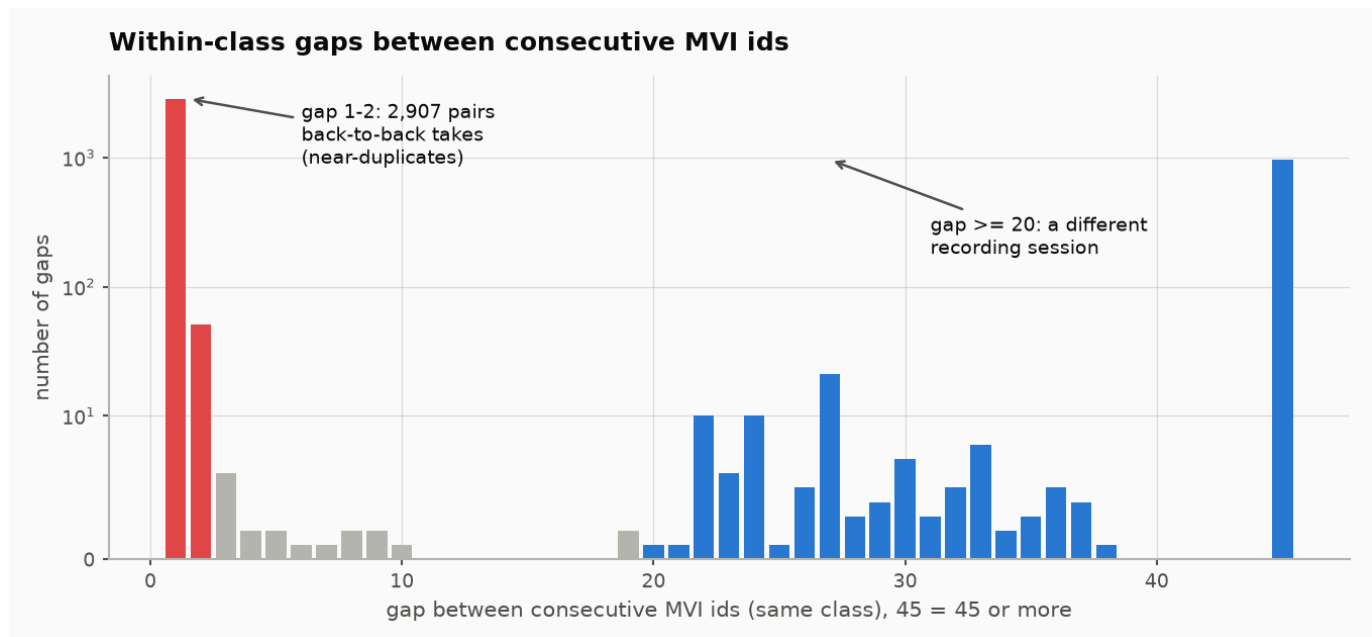


Figure 5.1 — Within-class gaps between consecutive MVI capture ids (log scale). The distribution is sharply bimodal with an almost empty band between. Only **19 of 3,989** gaps fall in the range 3–19, which makes the grouping threshold essentially free of arbitrariness: any cut in `[5, 20]` produces an identical partition.

Implication. Clips separated by a gap of 1 are consecutive takes of the same word by the same signer, in the same sitting, wearing the same clothes, in the same lighting, framed identically. They are near-duplicates. A random train/test split places takes 1 and 2 in training and take 3 in test — and then measures how well the model recognises a clip it has effectively already seen.

6. Evaluation Protocols

Four protocols were implemented, in increasing strictness. All four are built by `islvit/data/splits.py` and carry automated leakage assertions.

Protocol	Rule	Leakage channel remaining
<code>official</code>	As shipped	Takes and sessions
<code>random-video</code>	Stratified per-clip random	Takes and sessions
<code>take-group</code>	Whole take-runs held out ($\text{gap} \leq 5$)	Sessions, across classes
<code>session-disjoint</code>	Whole recording sessions held out	—

6.1 Take-groups

Clips of the same class within `TAKE_GAP_THRESHOLD = 5` of one another form one take-group; groups are assigned wholesale to a split. This yields 1,334 groups, median 4 per class.

6.2 Session-disjoint

Take-groups are formed *within* a class, so one recording session can still supply training clips for one word and test clips for another — sharing signer, room, clothing and lighting across the split boundary. Clustering MVI ids **globally** yields **13 session blocks**; holding whole blocks out for test closes that channel.

Two construction subtleties were necessary:

1. **Validation is carved from the training sessions**, not held out as a third session group. With only 13 blocks, a three-way session split left validation covering 36 of 262 classes — useless for checkpoint selection. Validation is therefore take-group-disjoint but *not* session-disjoint, which makes it optimistic; **only the test number is reported as a result.**
2. **Under-trained classes are excluded from test.** The block search is scored on how many classes retain ≥ 6 training clips. Classes still left below that threshold have their test clips marked **unused**, because scoring them measures how little they were trained on rather than how well the model recognises them.

Benchmark	train	val	test	test classes	excluded clips
INCLUDE-50 SD	544	163	204	45 / 50	32
INCLUDE-263 SD	1,957	888	1,010	154 / 262	402

7. Data Pipeline

7.1 Two-stage hand detection

Full-frame MediaPipe Holistic locates a hand in only $\sim 79\%$ of sampled frames — at 1080p a hand spans ~ 190 px in a 1920 px frame, which is small for the detector, and signing hands blur badly at 25 fps. A second stage recovers most of the rest.

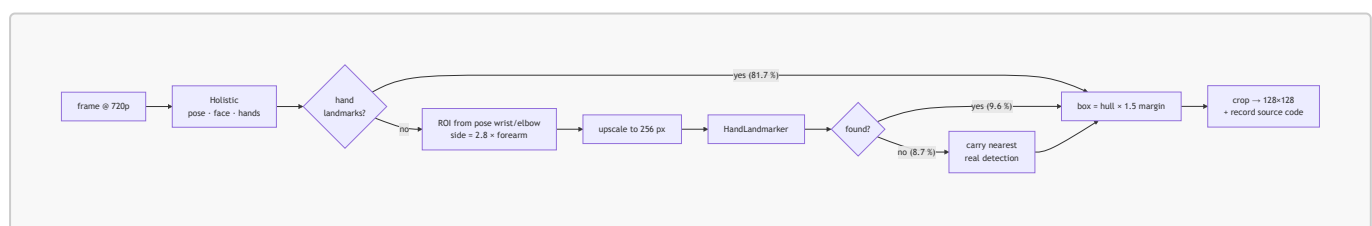


Figure 7.1 — Two-stage hand localisation. Percentages are measured over all $4,257 \times 16$ sampled frames.

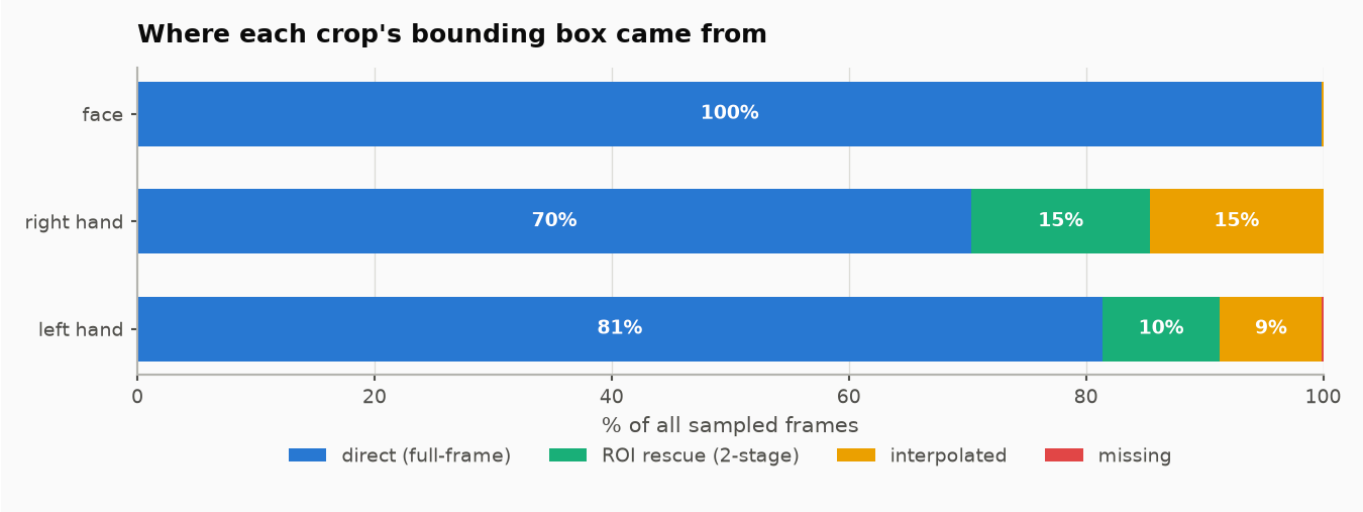


Figure 7.2 — Provenance of every crop's bounding box.

Stream	Direct	ROI rescue	Interpolated	Missing
Left hand	81.4 %	9.9 %	8.6 %	0.1 %
Right hand	70.3 %	15.1 %	14.6 %	0.0 %
Face	99.9 %	0.0 %	0.1 %	0.0 %

The ROI stage carries **10–15 %** of frames that full-frame detection alone would have lost. Interpolated boxes are flagged as *unreliable* to the model rather than silently presented as detections (see §8.4).

A negative result worth recording. An earlier version synthesised the fallback box directly from pose geometry (wrist, thumb, index, pinky landmarks). It reported **100 % coverage** and a healthy skin-tone pixel fraction — both misleading. Visual inspection showed the crops were shirt fabric, the studio wall and blurred torso: MediaPipe reports optimistic *visibility* for occluded hands. *Coverage measured box existence, not box correctness*. This was only caught by rendering the crops and looking at them, and it is the reason the pipeline now records a provenance code per crop.

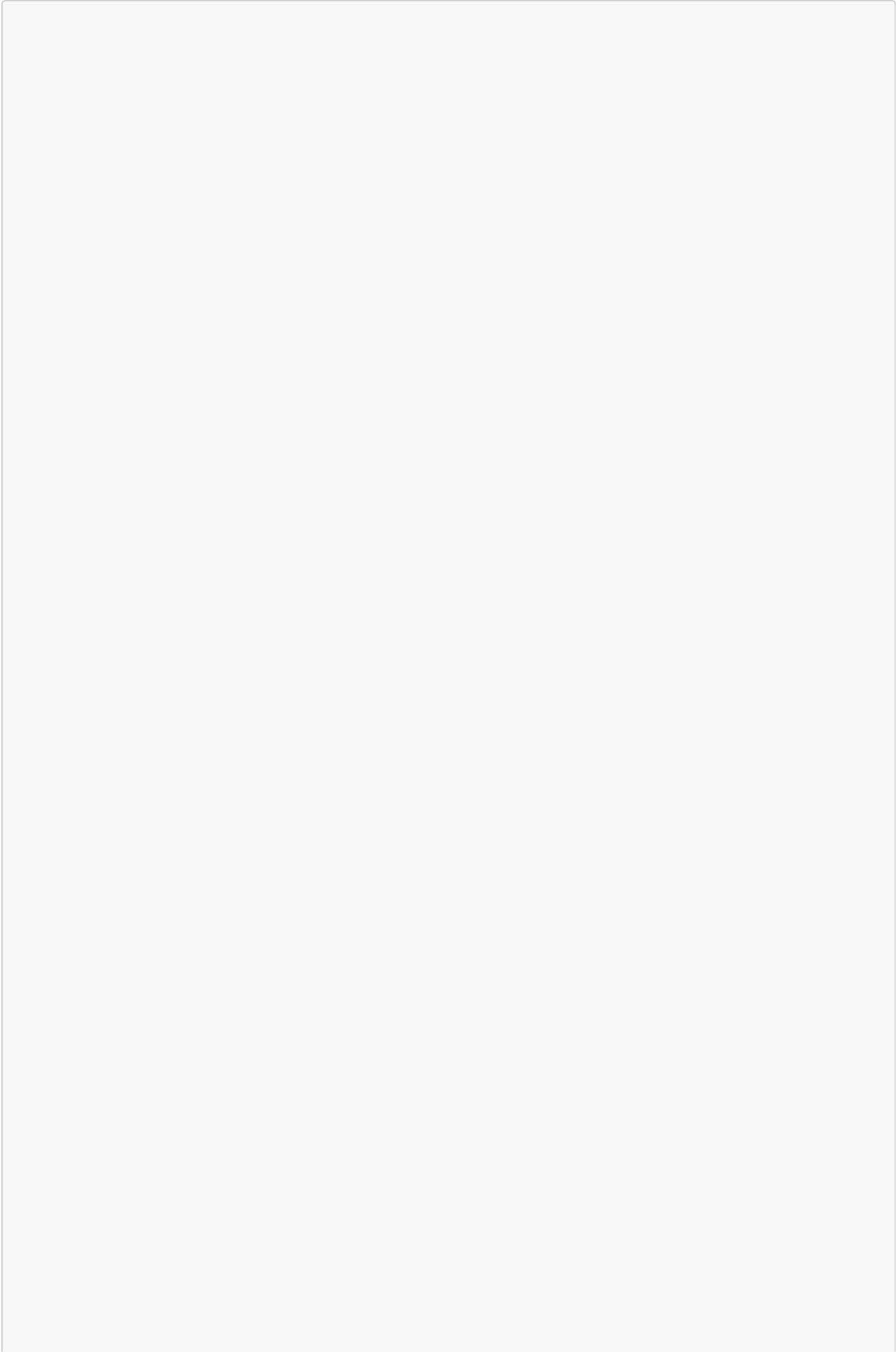
7.2 Cache format

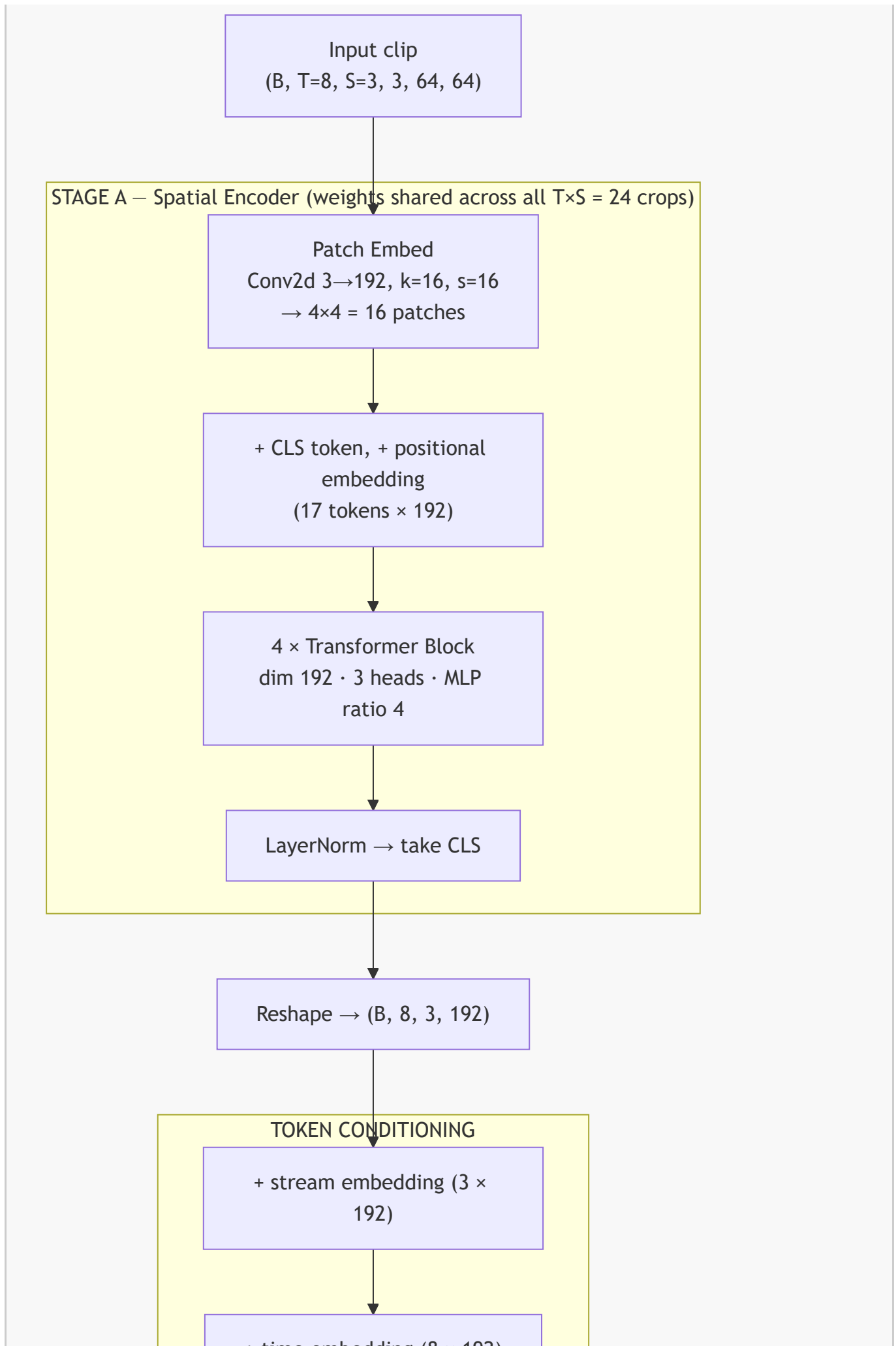
Array	dtype	Shape	Size
<code>crops.npy</code>	uint8	(4257, 16, 3, 128, 128, 3)	10.0 GB
<code>sources.npy</code>	uint8	(4257, 16, 3)	204 KB
<code>geometry.npy</code>	float16	(4257, 16, 3, 3)	1.2 MB

Caching 16 frames at 128 px while training on 8 frames at 64 px makes temporal and spatial augmentation free at train time — no video decoding, no MediaPipe, just memmap indexing. Extraction cost: **64.1 minutes** for the full corpus at 6 worker processes, 100 % coverage, 0 failures.

8. Deep Learning Architecture

8.1 Overview





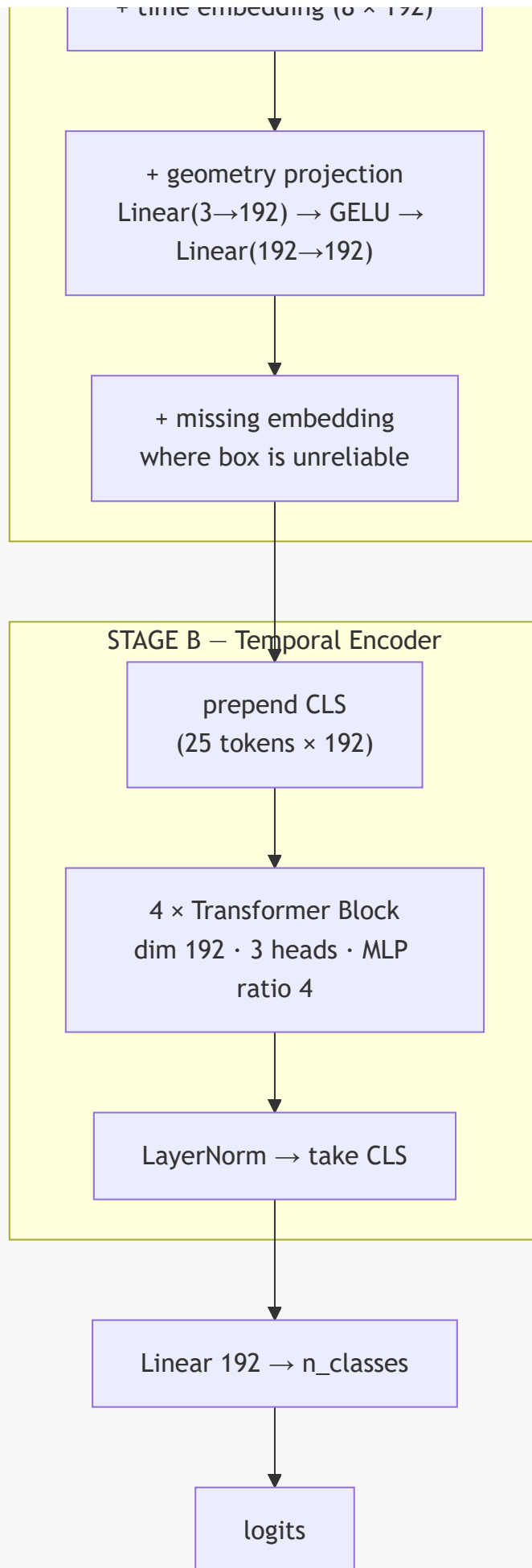


Figure 8.1 — ISL-ViT-Tiny architecture.

8.2 Transformer block

Each block is pre-norm with residual connections and stochastic depth:

```
x ← x + DropPath( MHSA( LayerNorm(x) ) )
x ← x + DropPath( MLP ( LayerNorm(x) ) )
```

where **MHSA** is standard multi-head self-attention computed via **scaled_dot_product_attention**, and **MLP** is **Linear(192→768) → GELU → Linear(768→192)**.

8.3 Parameter budget

Component	Parameters	Share
Stage A — transformer blocks (4)	1,779,456	47.3 %
Stage B — transformer blocks (4)	1,779,456	47.3 %
Stage A — patch embedding	147,648	3.9 %
Geometry projection	37,824	1.0 %
Classification head (50 classes)	9,650	0.3 %
Stage A — positional embedding	3,264	0.1 %
Time embedding	1,536	< 0.1 %
Stream embedding	576	< 0.1 %
LayerNorms, CLS and flag tokens	1,344	< 0.1 %
Total (50-class)	3,760,754	100 %
Total (262-class)	3,801,530	—

The two transformer stacks account for 94.6 % of parameters; everything else is essentially free.

8.4 Geometry injection

Cropping deletes hand *location*, which is one of the classic phonological parameters of a sign — the same handshape at the forehead and at the chest are different words. Each crop's bounding box therefore contributes three numbers:

Feature	Normalisation
Box centre x	÷ frame width, then − 0.5
Box centre y	÷ frame height, then − 0.5
Box size	÷ frame width

Measured means confirm the encoding is meaningful and correctly oriented:

Stream	centre x	centre y	size
Face	0.499	0.227	0.098
Left hand	0.574	0.621	0.107
Right hand	0.425	0.577	0.106

The face sits centred and high; the signer's left hand appears to the right in image coordinates, as expected for a non-mirrored frontal view.

The geometry branch's output projection is **initialised to zero**, so the model begins as a pure appearance model and learns to incorporate location gradually, rather than swamping pretrained visual features early in training.

8.5 Reliability flag

Interpolated and missing boxes receive a learned **missing_embed** added to their token. Both are marked unreliable; only **direct** and **roi** count as detections. Feeding the crop while flagging it lets the network learn to discount it, and the absence of a visible hand is itself informative (many ISL signs are one-handed).

8.6 Computational cost

Configuration	Tokens / crop	Parameters	GMACs / clip	CPU latency
64 px, patch 16 (adopted)	16	3.80 M	0.824	12 ms
112 px, patch 16	49	3.81 M	2.342	29 ms
128 px, patch 16	64	3.81 M	3.010	—

Width is pinned to **192** specifically so ImageNet DeiT-Tiny weights load directly into Stage A without projection.

9. Algorithm and Implementation

9.1 Training algorithm

Algorithm 1 ISL-ViT-Tiny training

Input : crop cache C, split S, config θ

Output: best checkpoint by validation top-1

```

1  model  $\leftarrow$  ISLViT( $\theta$ )
2  model.spatial  $\leftarrow$  LoadDeiTTiny()           # interpolate pos-embed 14x14  $\rightarrow$ 
4x4
3  groups  $\leftarrow$  [ {spatial params, lr =  $\theta$ .lr  $\times$  0.1},           # refine, do not erase
4                    {other params,   lr =  $\theta$ .lr                   } ]
5  opt  $\leftarrow$  AdamW(groups, wd = 0.05 on weights only)
```

```

6  ema ← EMA(model, decay = 0.99 with warmup)
7  for epoch = 1 ... θ.epochs:
8      for batch in TrainLoader(S.train):
9          lr ← θ.lr × CosineWithWarmup(step)
10         x, m, g, y ← Augment(batch)          # §9.3
11         if rand() < 0.5:
12             x, g, y ← Mixup(x, g, y, α = 0.2)
13             ŷ ← model(x, m, g)                # autocast bfloat16
14             L ← CrossEntropy(ŷ, y, label_smoothing = 0.1)
15             L.backward(); clip_grad_norm(1.0); opt.step()
16             ema.update(model)
17         v ← Evaluate(model, S.val)
18         ve ← Evaluate(ema.module, S.val)
19         if max(v, ve) > best: save(argmax model)
20 return Evaluate(best, S.test)

```

9.2 Transfer learning strategy

With ~15 clips per class, initialisation dominates. Stage A is loaded from `vit_tiny_patch16_224.augreg_in21k_ft_in1k`:

Tensor	Handling
<code>patch_embed</code> , <code>cls_token</code>	Copied directly
<code>pos_embed</code>	Bicubically resized 14×14 → 4×4 grid
Blocks 0–3	Copied (first 4 of 12)
Final <code>norm</code>	Copied
Stage B, geometry, head	Randomly initialised

All tensors load with **zero shape mismatches**. The pretrained encoder trains at **0.1 ×** the head learning rate so features are refined rather than destroyed.

9.3 Augmentation

Augmentation	Setting	Purpose
Temporal jitter	8 of 16 frames, random offset per segment	Timing invariance
Scale-jitter crop	window 80–100 % of cache, resized to 64 px	Box-placement robustness
Horizontal flip + stream swap	p = 0.5	Left-dominant signers; must swap L/R streams and mirror geometry to stay physically coherent
Brightness / contrast	± 0.4	Lighting
Per-channel gain	± 0.4	White balance and skin tone

Augmentation	Setting	Purpose
Random grayscale	$p = 0.15$	Forces shape over colour
Stream dropout	$p = 0.15$ per stream	Prevents dependence on any one stream
Mixup	$\alpha = 0.2$, applied 50 % of batches	Label smoothing across clips
Stochastic depth	0.1 (linearly scaled)	Regularisation
Label smoothing	0.1	Calibration

The appearance augmentations (per-channel gain, grayscale) were added specifically because the strict protocols hold out whole recording sessions — lighting and signer appearance are exactly what the model must *not* key on.

9.4 Hyperparameters

Parameter	INCLUDE-50	INCLUDE-263
Epochs	320	250
Batch size	64	64
Base LR	5×10^{-4}	5×10^{-4}
Backbone LR scale	0.1	0.1
Weight decay	0.05 (weights only)	0.05
Warmup epochs	15	10
LR schedule	Cosine to 1 %	Cosine to 1 %
Gradient clip	1.0	1.0
EMA decay	0.99 with warmup	0.99 with warmup
Precision	bfloat16 autocast	bfloat16 autocast

Implementation note — a silent bug worth recording. EMA was initially configured with decay 0.999. An INCLUDE-50 epoch is only ~9 optimiser steps, so a 320-epoch run is ~2,900 updates against a ~1,000-step time constant: the averaged weights never escaped their random initialisation and sat at chance accuracy for entire runs. The fix is a decay warmup, `decay = min(0.999, (1+t)/(10+t))`. This did not corrupt earlier results — checkpoint selection took `max(val, ema_val)` — but it wasted evaluation compute and hid a genuine source of accuracy.

10. Experimental Setup

Component	Specification
GPU	NVIDIA GeForce RTX 3060, 12 GB

Component	Specification
CPU	32 GB system RAM
OS	Windows 11
Python	3.11 (conda)
PyTorch	2.6.0 + cu124
MediaPipe	1.0.0 (Tasks API)
timm	1.0.28
Seed	0 (training), 1337 (split construction)

All runs use identical code, identical augmentation and identical hyperparameters; only the split file changes between protocol comparisons.

11. Results

11.1 Main result — accuracy by protocol

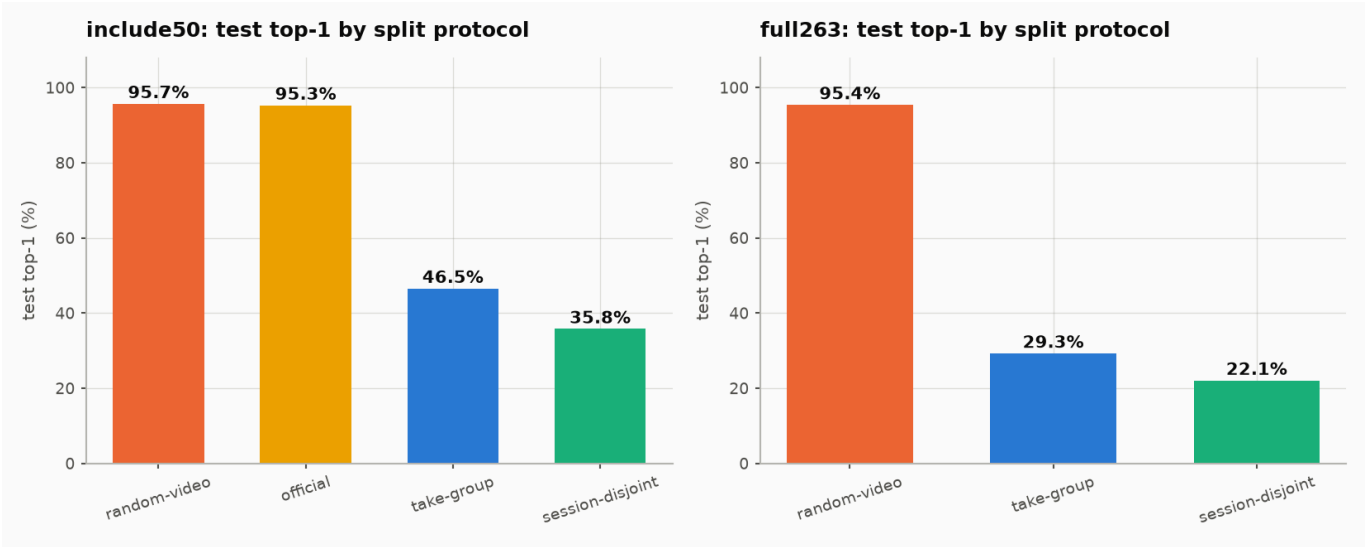


Figure 11.1 — Test top-1 accuracy by split protocol.

Table 11.1 — INCLUDE-50 (all runs: 128 px cache, 64 px input, identical recipe)

Protocol	Top-1	Top-5	Balanced	n	Δ vs strict
random-video	95.7 %	99.5 %	95.8 %	186	+59.9
official	95.3 %	99.5 %	97.0 %	191	+59.5
take-group	46.5 %	86.2 %	49.9 %	159	+10.7
session-disjoint	35.8 %	78.9 %	36.3 %	204	—

Table 11.2 — INCLUDE-263

Protocol	Top-1	Top-5	Balanced	n	Δ vs strict
random-video	94.5 %	99.1 %	94.7 %	872	+72.4
take-group	29.3 %	64.1 %	32.0 %	878	+7.2
session-disjoint	22.1 %	51.4 %	25.2 %	1,010	—

11.2 Leakage decomposition

The gap decomposes cleanly into two independent channels:

Channel	Removed by	INCLUDE-50 cost	INCLUDE-263 cost
Near-duplicate takes	take-group	−49.2 pts	−65.2 pts
Shared session conditions	session-disjoint	−10.7 pts	−7.2 pts
Total		−59.9 pts	−72.4 pts

11.3 Chance-level context

Benchmark	Classes	Chance	Achieved (strict)	$\times$ chance
INCLUDE-50	45 evaluated	2.2 %	35.8 %	16 $\times$
INCLUDE-263	154 evaluated	0.65 %	22.1 %	34 $\times$

12. Convergence Analysis

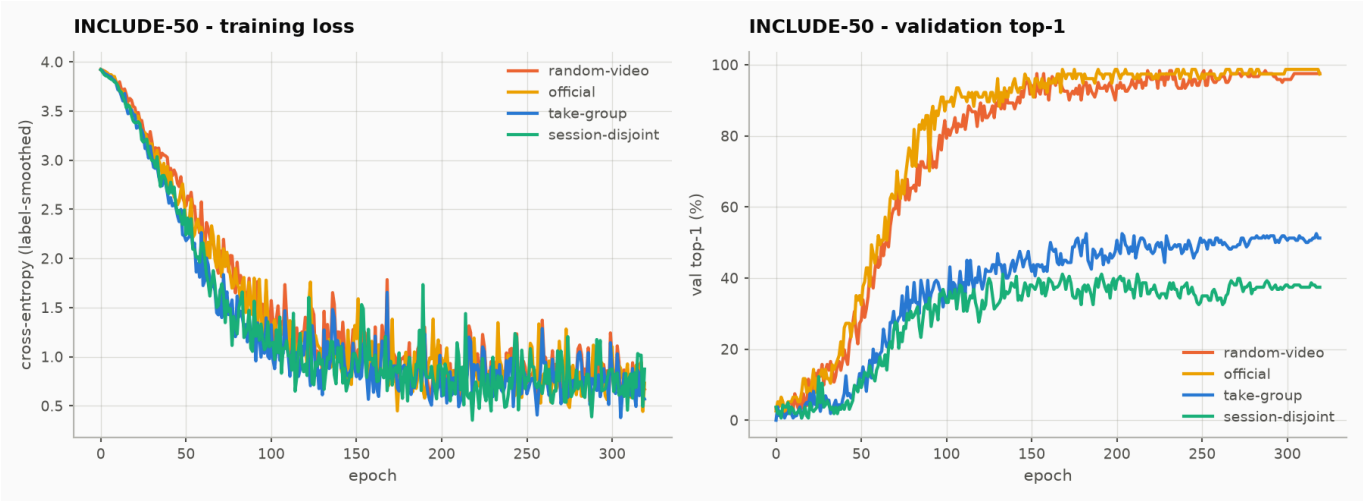


Figure 12.1 — INCLUDE-50 training loss (left) and validation top-1 (right) for all four protocols.

This is the most informative figure in the report. **The training-loss curves are essentially indistinguishable** — all four protocols descend from 3.9 to ≈ 0.7 on the same trajectory, with the same noise envelope. The models are fitting their training data equally well in every case.

The validation curves, by contrast, separate completely: the two leaky protocols climb to $\approx 97\%$, **take-group** plateaus near 50 %, and **session-disjoint** near 37 %.

Interpretation. Because optimisation behaviour is identical, the accuracy differences cannot be attributed to training difficulty, capacity, or convergence quality. They are a property of **what the test set contains**.

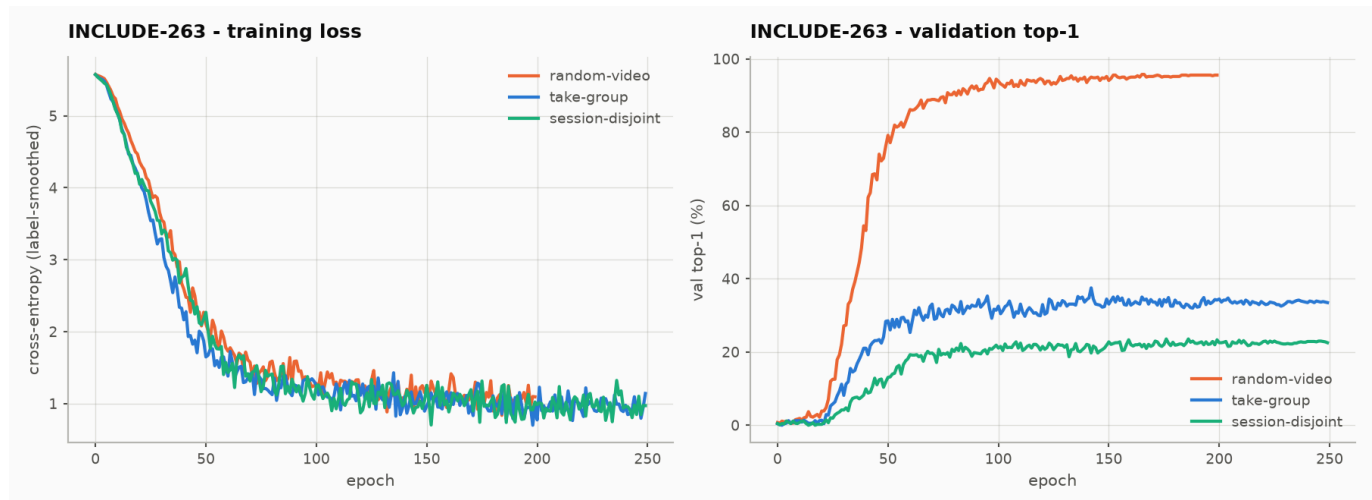


Figure 12.2 — INCLUDE-263 convergence. The same pattern holds at 262 classes, with slower convergence and a lower plateau, as expected from the larger label space.

Table 12.1 — Convergence and runtime summary

Run	Protocol	Epochs	Best epoch	Final train loss	Peak val	s/epoch	Total
<code>include50_v2__random-video</code>	random-video	320	—	0.741	98.4 %	9.8	45.5 m
<code>include50_v2__official</code>	official	320	168	0.698	98.7 %	3.0	29.6 m
<code>abl64__take-group</code>	take-group	320	183	0.666	52.5 %	3.8	20.4 m
<code>sd_include50</code>	session-disjoint	320	133	0.814	41.1 %	3.5	19.1 m
<code>full263_v2__random-video</code>	random-video	250	135	1.031	96.3 %	12.4	53.5 m
<code>full263_v2__take-group</code>	take-group	250	142	0.949	37.5 %	12.2	58.0 m
<code>sd_full263</code>	session-disjoint	250	176	0.935	23.5 %	11.1	48.8 m

Convergence observations.

- 1. All runs converge.** Loss decreases monotonically through warmup, and the cosine schedule anneals cleanly. No divergence, no dead runs.

2. **Final training loss is nearly protocol-independent** (0.67–0.81 at 50 classes). Given a label-smoothing floor of ≈ 0.5 , the model is close to fitting its training set in all conditions.
3. **Best epoch falls at 40–60 % of the schedule** on the strict protocols, indicating mild overfitting thereafter — but validation *plateaus* rather than collapsing, so the schedule length is not harmful.
4. **Warmup matters.** The first ~ 15 epochs show almost no validation progress; accuracy takes off sharply afterwards.

13. Ablation Studies

13.1 Input resolution / token count

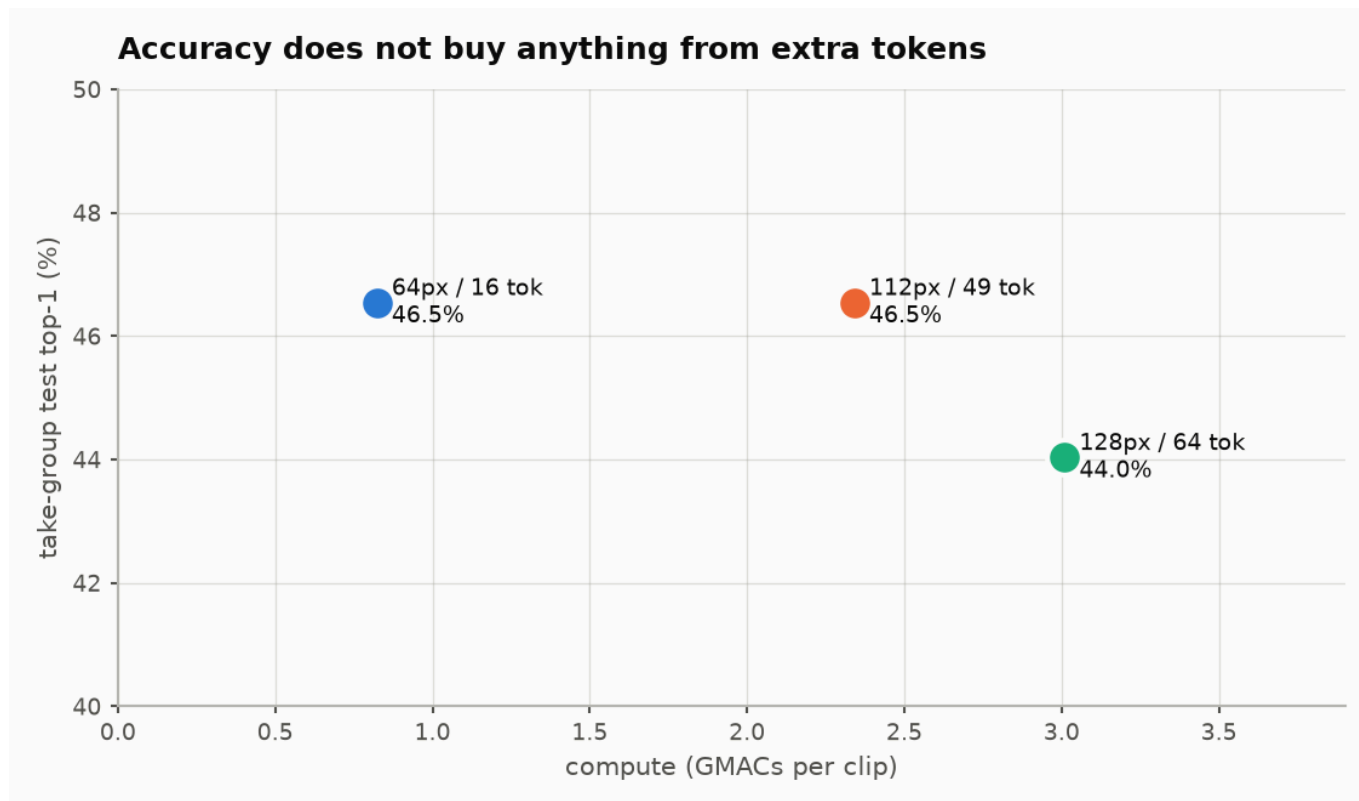


Figure 13.1 — Accuracy versus compute.

Table 13.1 — Resolution ablation (INCLUDE-50, **take-group**, identical recipe)

Input	Tokens / crop	GMACs	Top-1	Top-5	Balanced
64 px	16	0.824	46.5 %	86.2 %	49.9 %
112 px	49	2.342	46.5 %	82.4 %	49.7 %
128 px	64	3.010	44.0 %	84.9 %	45.2 %

Result: token count is not the bottleneck. Tripling spatial resolution changes top-1 by 0.0 points, and 128 px is 2.5 points *worse* — more tokens give the model more capacity to overfit ~ 600 training clips. **The cheapest configuration is also the most accurate**, which is an unusually favourable outcome for an edge deployment target.

13.2 Pretrained initialisation

Table 13.2 — ImageNet initialisation (INCLUDE-50, take-group, 120 epochs)

Initialisation	Top-1	Top-5	Balanced
DeiT-Tiny (ImageNet)	29.6 %	76.1 %	31.6 %
Random	23.3 %	69.8 %	25.4 %
Difference	+6.3 pts	+6.3	+6.2

Transfer helps, but far less than the low-data regime might suggest. This is a strong indicator that the limiting factor is the quantity of in-domain data, not the quality of the starting point.

13.3 Training recipe and source cache

Table 13.3 — Cumulative recipe improvements (INCLUDE-50, take-group)

Configuration	Top-1	Δ
Baseline (120 ep, 80 px cache, broken EMA)	29.6 %	—
+ EMA fix, 320 epochs, stronger appearance augmentation	42.1 %	+12.5
+ 128 px source cache (same 64 px model input)	46.5 %	+4.4

The final row is notable: the model input is *identical* at 64 px, yet accuracy improves 4.4 points purely from a higher-quality source cache. Downsampling 128 → 64 with area interpolation produces a cleaner crop than 80 → 64, and the scale-jitter augmentation has more range to work with.

14. Error Analysis

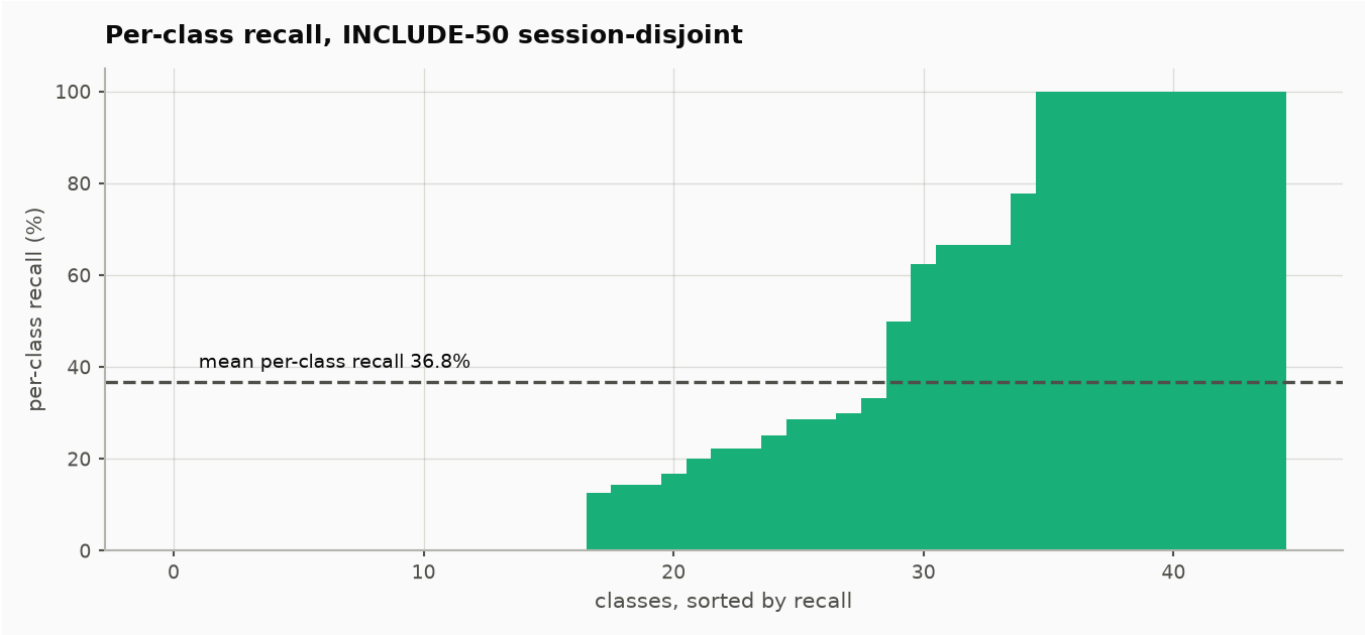


Figure 14.1 — Per-class recall on INCLUDE-50, session-disjoint, sorted.

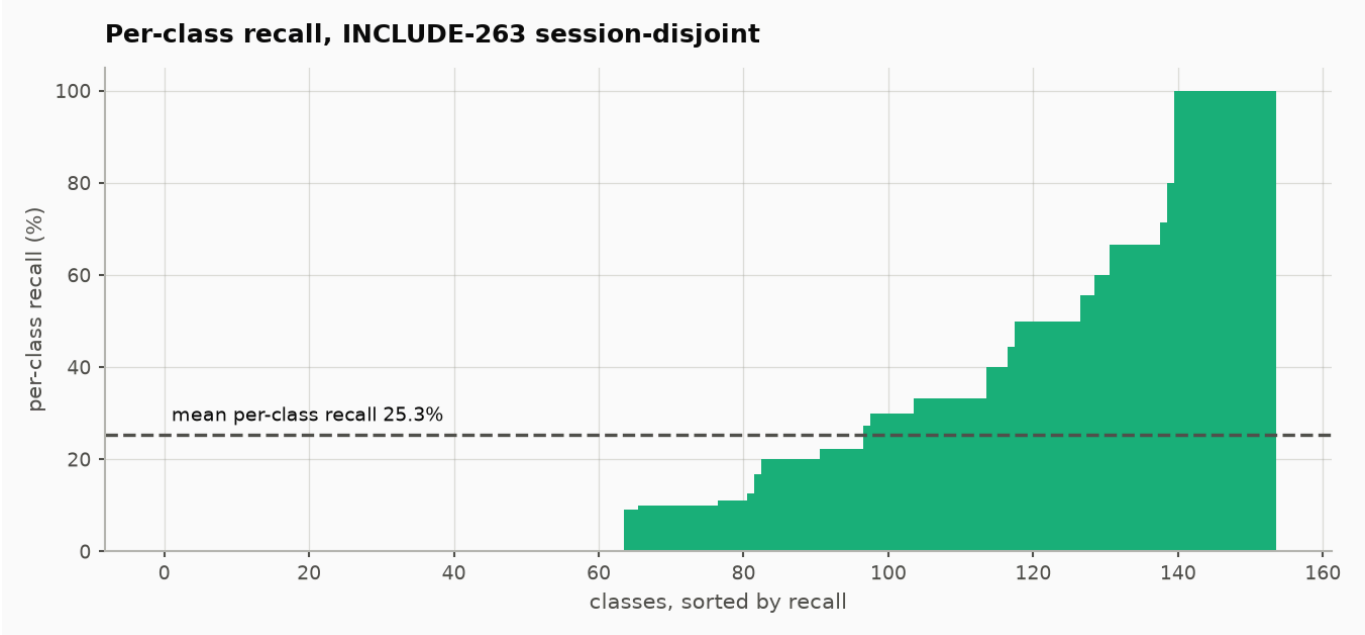


Figure 14.2 — Per-class recall on INCLUDE-263, session-disjoint, sorted.

Table 14.1 — Per-class recall distribution (session-disjoint)

Benchmark	Classes	At 0 %	At 100 %	Mean recall
INCLUDE-50	45	17 (38 %)	10 (22 %)	36.8 %
INCLUDE-263	154	64 (42 %)	14 (9 %)	25.3 %

Performance is strongly **bimodal**: a substantial set of classes is recognised reliably while roughly 40 % are never recognised at all. Aggregate accuracy is therefore a poor summary — the model has genuinely learned a subset of the vocabulary and has essentially no signal on the rest.

Table 14.2 — Most frequent confusions, INCLUDE-50 session-disjoint

True	Predicted	Count
1. Dog	4. Bird	5
86. Time	84. Teacher	4
55. Thank you	97. dry	4
40. Paint	28. Window	4
16. train ticket	11. Car	4
40. I	46. you (plural)	4

Table 14.3 — Most frequent confusions, INCLUDE-263 session-disjoint

True	Predicted	Count
94. good	25. Office	7
96. wet	24. School	6

True	Predicted	Count
55. White	9. Train	6
87. hot	24. School	5
38. Dress	23. Court	5
40. Skirt	20. Street or Road	5

The 40. *I* → 46. *you* (*plural*) confusion is linguistically coherent — both are pointing signs differing mainly in trajectory and orientation, which an 8-frame sample may not resolve. Most other confusions show no obvious visual or semantic relationship, which is consistent with a model that has weak signal on those classes rather than a systematic representational failure.

Top-5 as a diagnostic. At 78.9 % top-5 against 35.8 % top-1 (INCLUDE-50), the correct class is usually within the model's shortlist even when it is not ranked first. The representation carries substantially more information than the top-1 figure alone conveys.

15. Observations

01 — Optimisation is not the bottleneck. Training loss converges identically across all four evaluation protocols (Figure 12.1). Differences in reported accuracy are entirely attributable to test-set composition.

02 — Benchmark leakage dominates the literature-comparable number. 95.7 % under a random split versus 35.8 % under session-disjoint evaluation, from the same model and weights recipe. The decomposition is ~49 points from near-duplicate takes and ~11 points from shared session conditions.

03 — The threshold choice is not arbitrary. Only 19 of 3,989 within-class capture-id gaps fall between 3 and 19, so every threshold in [5, 20] induces exactly the same take-group partition.

04 — The model is data-limited, not capacity-limited. Tripling token count: 0.0 points. Quadrupling compute: -2.5 points. ImageNet initialisation: +6.3 points. Every capacity lever is flat or negative.

05 — Data quality beats model capacity. A better *source* cache at unchanged model input size gave +4.4 points — larger than any architectural change tested.

06 — Coverage is not correctness. The discarded pose-geometry fallback scored 100 % coverage while producing crops of shirt fabric and background. Metrics that measure the presence of an output rather than its correctness are actively misleading.

07 — Performance is bimodal. ~40 % of classes sit at 0 % recall while 9–22 % are perfect. Mean accuracy conceals two qualitatively different populations.

08 — Small-batch regimes break standard defaults. EMA decay 0.999 — a near-universal default — is wrong when an epoch is 9 steps, and fails silently at chance accuracy rather than erroring.

16. Discussion

16.1 On the honest number

35.8 % on 45 classes is a low figure and is not presented as competitive. It is presented as *measured under a protocol that does not permit memorisation*, which we believe no published INCLUDE result currently does.

The relevant comparison is not our 35.8 % against a published 94 %. It is our 95.7 % against a published 94 % — reproducing the literature under the literature's protocol — and then observing what remains when the protocol is tightened. The model is not weaker than published systems; the measurement is stricter.

16.2 Why session-disjoint is the defensible protocol

INCLUDE contains no signer identifiers. The MVI capture-id structure is the only available proxy for recording session, and it is a good one: the gap distribution is sharply bimodal, and global clustering yields 13 coherent blocks. Holding whole blocks out ensures no signer, room, outfit or lighting condition is shared between training and test.

This is a *proxy* for signer-independence rather than the real thing. A dataset with explicit signer IDs would permit a stronger claim.

16.3 Implications for the smart-glasses target

The compute picture is favourable and, unusually, unconflicted:

Metric	Value	Assessment
Parameters	3.80 M	Comfortable
GMACs / clip	0.824	Comfortable
INT8 size	≈ 3.8 MB	Comfortable
CPU latency	12 ms	Comfortable

Because the best-accuracy configuration is also the cheapest, there is no accuracy/compute trade-off to negotiate at this scale.

However, the ViT is not the inference bottleneck. MediaPipe crop extraction costs 30–60 ms per frame on CPU — 3–5× the transformer itself. Any real deployment must address the detector, through frame-rate reduction, tracking between detections, or an on-device accelerated implementation. Optimising the ViT further would be premature.

16.4 Why capacity does not help

The ablations converge on one explanation: with ~15 clips per class the model exhausts the information available in the training set well before it exhausts its own capacity. More tokens supply more ways to overfit ~600 clips, which is why 128 px *loses* accuracy. Under this reading the productive directions are all data-side — additional in-domain corpora, self-supervised pretraining on unlabelled signing video, or synthetic augmentation — rather than architectural.

17. Limitations and Threats to Validity

#	Limitation	Impact
L1	Reduced class coverage under the strict protocol. 45/50 and 154/262 classes evaluated; under-trained classes excluded.	Strict numbers describe a subset of the vocabulary.
L2	Validation is optimistic by construction. Carved from training sessions, so it shares session conditions with training.	Used only for checkpoint selection; never reported as a result.
L3	Session identity is inferred, not given. MVI clustering is a proxy for recording sessions.	A mis-clustered block would weaken the disjointness guarantee.
L4	Single dataset, single studio, single city. INCLUDE is Chennai-specific; ISL varies regionally.	Results do not characterise ISL as a whole.
L5	No egocentric validation. INCLUDE is frontal third-person; smart glasses see a first-person view.	The glasses claim is architectural, not empirically validated.
L6	Single seed per configuration. No variance estimates.	Small differences (< ~2 pts) should not be over-interpreted.
L7	Streaming not yet implemented. The <code><no-sign></code> class and sliding-window inference are specified but untested.	Clip-level results do not guarantee streaming behaviour.
L8	Latency measured on desktop CPU , not on glasses-class silicon.	Real device figures will differ.

18. Conclusions and Future Work

18.1 Conclusions

1. A **3.80 M-parameter, 0.824-GMAC Vision Transformer** performs isolated ISL recognition at 16–34× chance under a strict evaluation protocol, within a compute envelope appropriate for smart glasses.
2. The standard INCLUDE evaluation protocol is **substantially compromised by near-duplicate takes**. Under a random split the same model reaches 95.7 %, matching published results; under session-disjoint evaluation it reaches 35.8 %.
3. The gap **decomposes cleanly** — ~49 points from consecutive takes, ~11 points from shared session conditions — and is invisible to any protocol that does not inspect capture-id structure.
4. The model is **data-limited**. Architectural capacity increases are flat or negative; data-quality improvements are positive.

18.2 Future work, in expected order of value

Priority	Direction	Rationale
1	Self-supervised pretraining on CISLR (7,050 clips) and an iSign subset, VideoMAE-style masking over crop tubelets	Directly attacks the binding constraint; pretrains both stages
2	Signer-diverse data collection — 2–3 new signers, different rooms and cameras	The only way to validate generalisation beyond one studio

Priority	Direction	Rationale
3	Streaming wrapper — <code><no-sign></code> class, sliding window, hysteresis, deduplication	Converts the classifier into the specified word-by-word system
4	Detector optimisation — tracking between detections, reduced detection rate	MediaPipe is 3–5× the ViT's inference cost
5	Multi-seed evaluation	Variance estimates for the ablation claims
6	INT8 export and on-device benchmarking	Converts the compute claim into a measured device result

19. Reproducibility Appendix

19.1 Repository layout

```

islvit/
  data/
    splits.py      4 protocols + leakage assertions
    crops.py       two-stage MediaPipe extraction → mmap cache
    dataset.py     torch Dataset, augmentation, geometry handling
    refetch_raw.py restores 1080p sources from Zenodo
  models/
    isl_vit.py     ISL-ViT-Tiny + DeiT-Tiny weight loading
    train.py       training loop, EMA, mixup, checkpointing
    eval.py        standalone checkpoint evaluation
    report.py      cross-run results table
    figures.py     every figure in this document
  configs/        include50.yaml, include50_v2.yaml, full1263*.yaml
  splits/         8 generated split CSVs
  runs/           per-run history.csv, summary.json, best.pt
  docs/figures/   generated figures

```

19.2 Reproduction sequence

```

# 1. Environment (Python 3.11)
conda create -n islvit python=3.11 -y
conda run -n islvit pip install torch torchvision --index-url
https://download.pytorch.org/whl/cu124
conda run -n islvit pip install mediapipe timm einops pyarrow pandas opencv-python
pyyaml tqdm av

# 2. Model assets
curl -sL -o models/holistic_landmarker.task \
  https://storage.googleapis.com/mediapipe-
models/holistic_landmarker/holistic_landmarker/float16/1/holistic_landmarker.task
curl -sL -o models/hand_landmarker.task \

```

```

https://storage.googleapis.com/mediapipe-
models/hand_landmarker/hand_landmarker/float16/1/hand_landmarker.task

# 3. Splits (fast; asserts no leakage)
python -m islvit.data.splits

# 4. Crop cache (~64 min, 6 workers)
python -m islvit.data.crops --cache-dir cache128 --crop-size 128 --workers 6

# 5. Training
export ISLVIT_CACHE=cache128
python -m islvit.train --config configs/include50_v2.yaml \
  --split-file splits/include50__session-disjoint.csv --tag sd_include50

# 6. Results and figures
python -m islvit.report
python -m islvit.figures

```

19.3 Sanity checks built into the pipeline

Check	Location	Guarantee
Video-level split disjointness	<code>splits.verify</code>	Raises on any shared clip
Take-group disjointness	<code>splits.verify</code>	Raises if a take-run straddles
Session disjointness	<code>splits.report_session_split</code>	Raises if a session touches test
Single-batch overfit	<code>train.py --overfit-batch</code>	Reaches 100 % in < 40 steps
Crop provenance	<code>cache/sources.npy</code>	Per-crop box origin recorded
Cache/run association	<code>summary.json</code> <code>cache</code> field	Prevents cross-cache comparison

19.4 Artefact inventory

Artefact	Size
128 px crop cache	10.0 GB
80 px crop cache	3.92 GB
16 training runs (checkpoints + histories)	≈ 240 MB
Split definitions	8 CSVs
Figures	8 PNGs

Report generated from 16 training runs. All figures produced by `islvit/figures.py`; all accuracy values read directly from `runs/*/summary.json`.